

Generative Adversarial Networks (GANs)

Have you ever seen them?

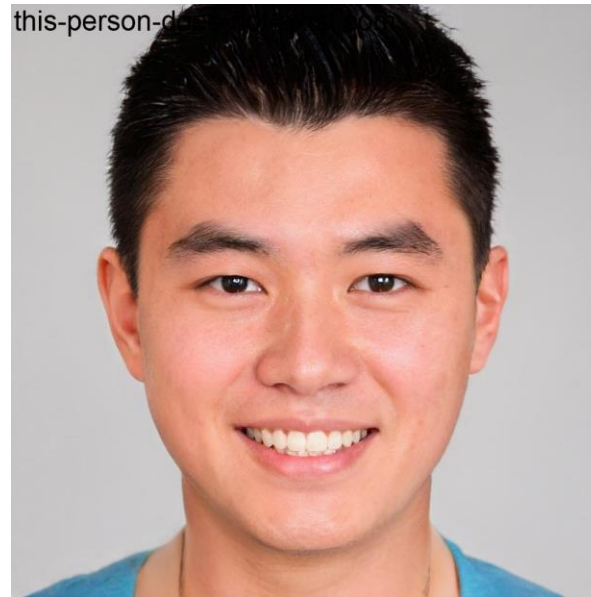


Fig: Images generated by a GAN (thispersondoesnotexist.com)

Is gravity required to ride a horse?

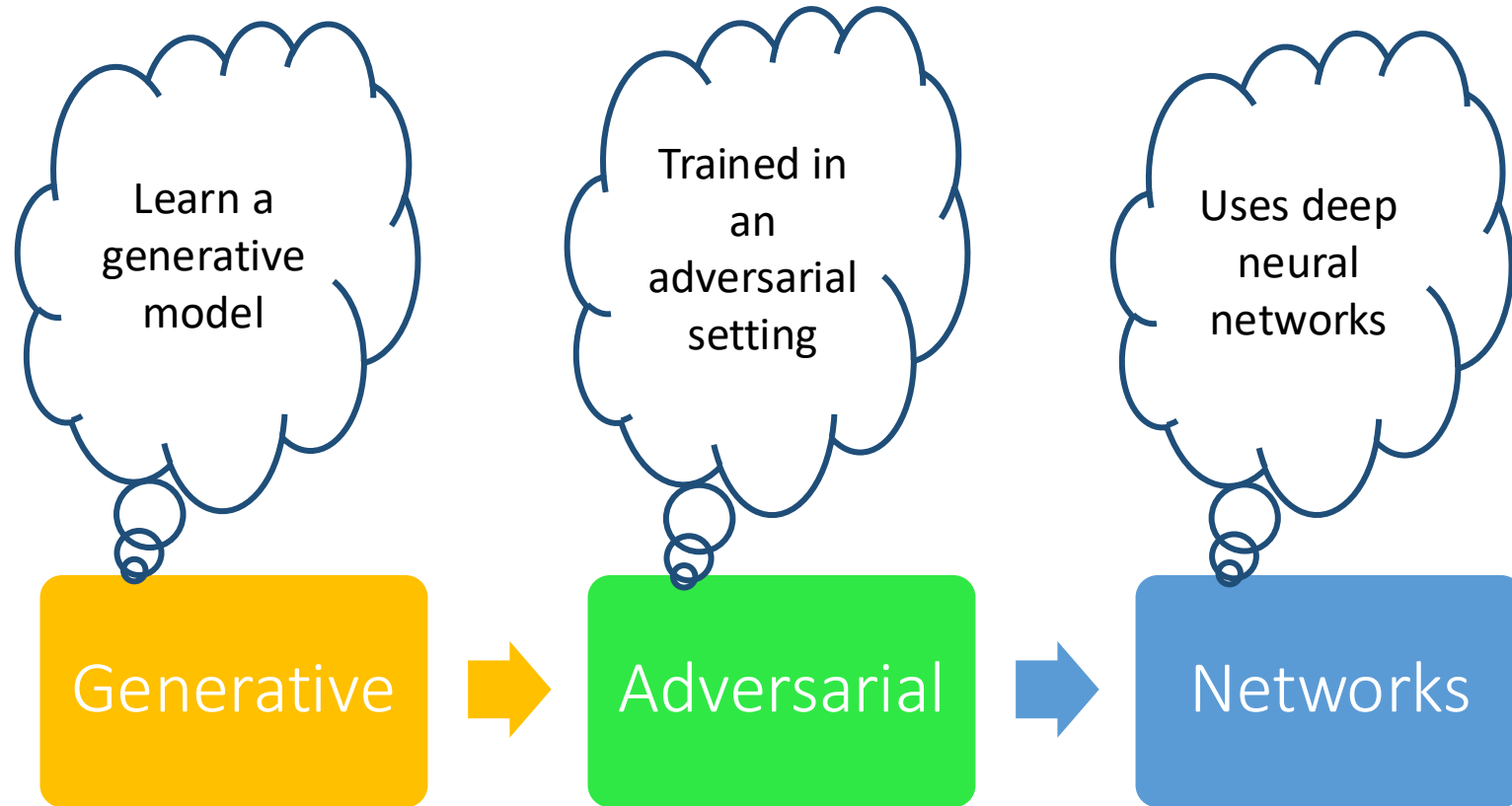


Fig: A photo of an astronaut riding a horse
(generated by DALL-E, a conditional GAN application).

Image-to-Image Translation with Conditional Adversarial Networks



What are GANs?



Generative Models

Discriminative models

- Can discriminate between different kinds of data instances.
- Capture the conditional probability $p(Y|X)$.

Generative models

- Can generate new data instances.
- Can capture the joint probability $p(X, Y)$, or just $p(X)$ if there are no labels.

Generative Models

- **Discriminative models-**

- Given an image 'X' predict a label 'Y'
- Estimates $P(Y|X)$

- **Limitations of discriminative models-**

- Can't model $P(X)$, i.e., the probability distribution from where 'X' is coming
- Can't generate new images

- **Generative models-**

- Can model $P(X)$
- Can generate new images

Why Generative Models?

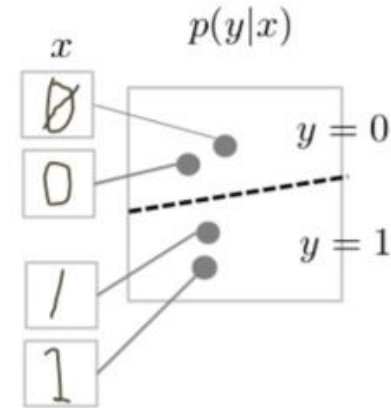
Discriminative Models:

- Discriminative models are designed to learn the decision boundary between different classes in a dataset.
- They focus on predicting the conditional probability of a class given input data.
- Examples of discriminative models include logistic regression, support vector machines (SVM), and neural networks with softmax activation in the output layer.
- Discriminative models are generally used for classification tasks.
- They require labeled data for training, where the input features and corresponding labels are used to learn the decision boundary.
- Discriminative models are known to be effective when the decision boundary is complex and non-linear.

Generative Models:

- Generative models are designed to learn the underlying distribution of the data in order to generate new data samples.
- They focus on modeling the joint probability of input features and labels or the unconditional probability of input features.
- Examples of generative models include Gaussian Mixture Models (GMM), Variational Autoencoders (VAEs), and Generative Adversarial Networks (GANs).
- Generative models can be used for tasks such as data generation, data augmentation, and anomaly detection.
- They can also be used for classification tasks by estimating the posterior probability of classes using Bayes' rule.
- Generative models require both labeled and unlabeled data for training, as they learn the underlying data distribution.
- Generative models are known to be effective when generating new data samples or when the available labeled data is limited.

• Discriminative Model



• Generative Model

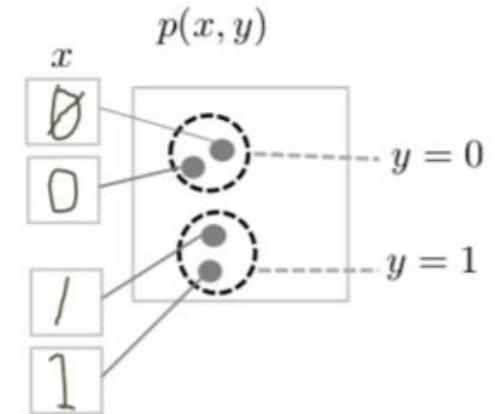


Fig: Discriminative and generative models of handwritten digits.
(Discriminative models focus on learning the decision boundary between classes, while generative models focus on learning the underlying data distribution to generate new data samples)

How do GANs learn?



As training progresses, the generator gets closer to producing output that can fool the discriminator:



Finally, if generator training goes well, the discriminator gets worse at telling the difference between real and fake. It starts to classify fake data as real, and its accuracy decreases.



High level architecture of GANs

- ❑ A generative adversarial network (GAN) has two parts:
 - The **generator** learns to generate plausible data. The generated instances become negative training examples for the discriminator.
 - The **discriminator** learns to distinguish the generator's fake data from real data. The discriminator penalizes the generator for producing implausible results.
- ❑ When training begins, the generator produces obviously fake data, and the discriminator quickly learns to tell that it's fake.
- ❑ As training progresses, the generator gets closer to producing output that can fool the discriminator.
- ❑ Finally, if generator training goes well, the discriminator gets worse at telling the difference between real and fake. It starts to classify fake data as real, and its accuracy decreases.

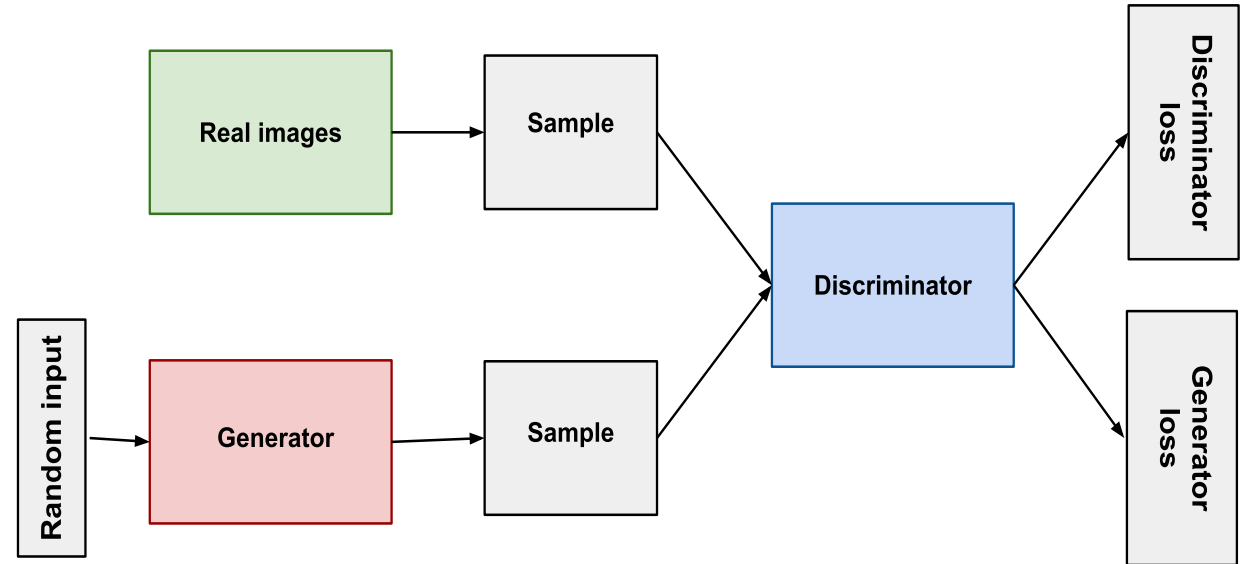


Fig: Overview of the generator-discriminator structure

Learning of a Discriminator

- Uses a classifier model to distinguish real data from the data created by the generator.

Discriminator Training Data-

- The discriminator's training data comes from two sources:
- **Real data** instances, such as real pictures of people. The discriminator uses these instances as positive examples during training.
- **Fake data** instances created by the generator. The discriminator uses these instances as negative examples during training.

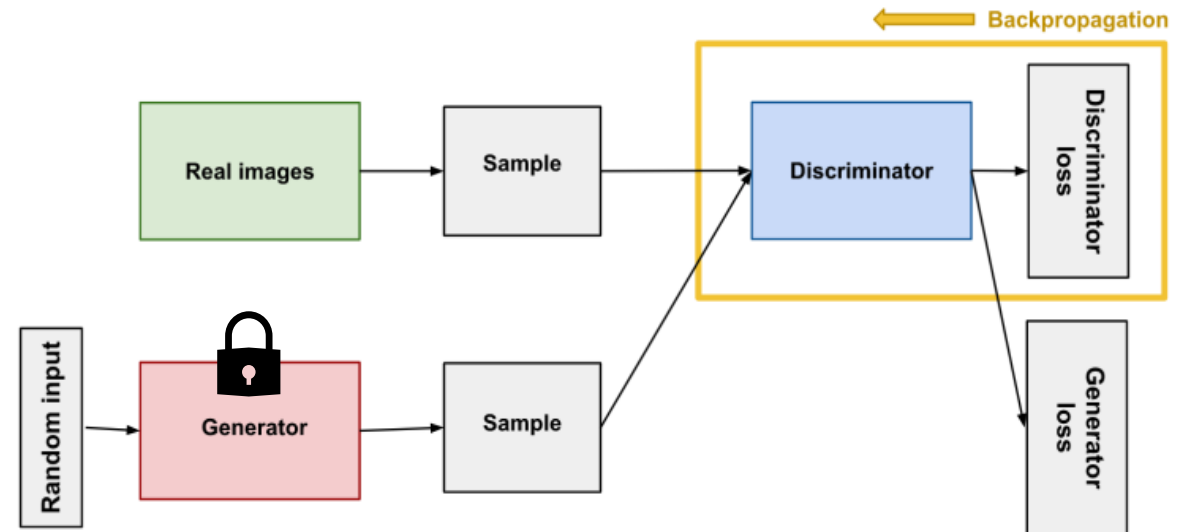


Fig: Overview of the learning process of a discriminator (generator's weights are locked)

Learning of a Discriminator(Continued)

❑ Training the Discriminator

- The discriminator connects to two loss functions. During discriminator training, the discriminator ignores the generator loss and just uses the discriminator loss. We use the generator loss during generator training, as described in the next slide.

❑ During discriminator training:

1. The discriminator classifies both real data and fake data from the generator.
2. The discriminator loss penalizes the discriminator for misclassifying a real instance as fake or a fake instance as real.
3. The discriminator updates its weights through backpropagation from the discriminator loss through the discriminator network.

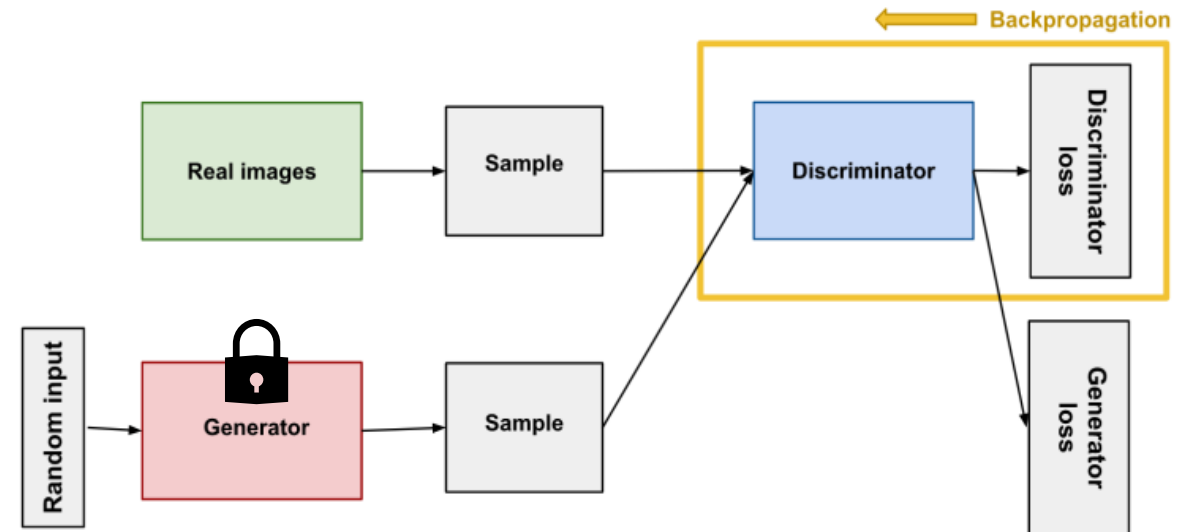
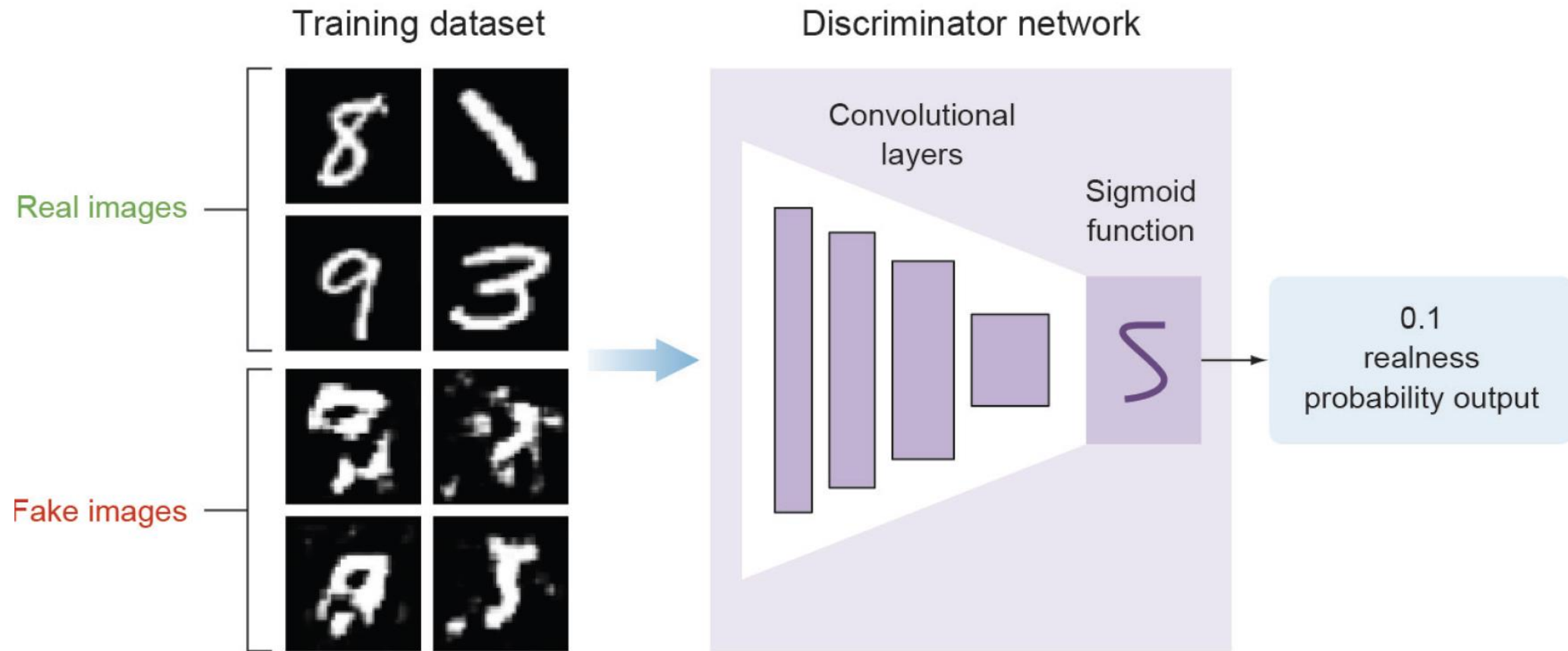


Fig: Overview of the learning process of a discriminator (generator's weights are locked)

Discriminator Model



Learning of a Generator

- ❑ The generator part of a GAN learns to create fake data by incorporating feedback from the discriminator. It learns to make the discriminator classify its output as real.
- ❑ Generator training requires tighter integration between the generator and the discriminator than discriminator training requires.
- ❑ The portion of the GAN that trains the generator includes:
 - random (noise) input
 - generator network, which transforms the random input into a data instance
 - discriminator network, which classifies the generated data
 - discriminator output
 - generator loss, which penalizes the generator for failing to fool the discriminator

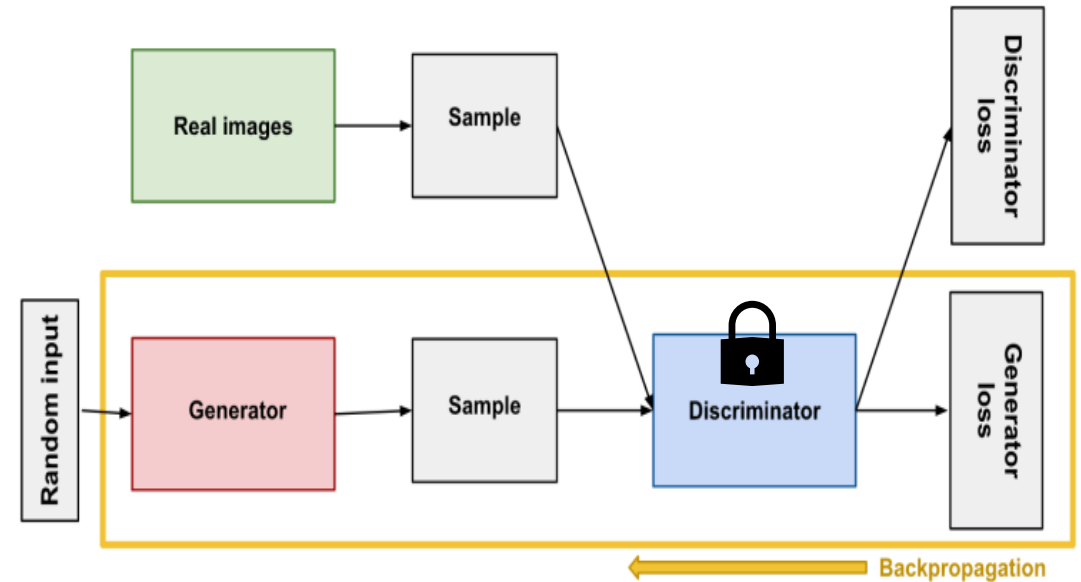


Fig: Overview of the learning process of a generator
(discriminator's weights are locked)

Learning of a Generator

Using the Discriminator to Train the Generator

1. Sample random noise.
2. Produce generator output from sampled random noise.
3. Get discriminator "Real" or "Fake" classification for generator output.
4. Calculate loss from discriminator classification.
5. Backpropagate through both the discriminator and generator to obtain gradients.
6. Use gradients to change only the generator weights.

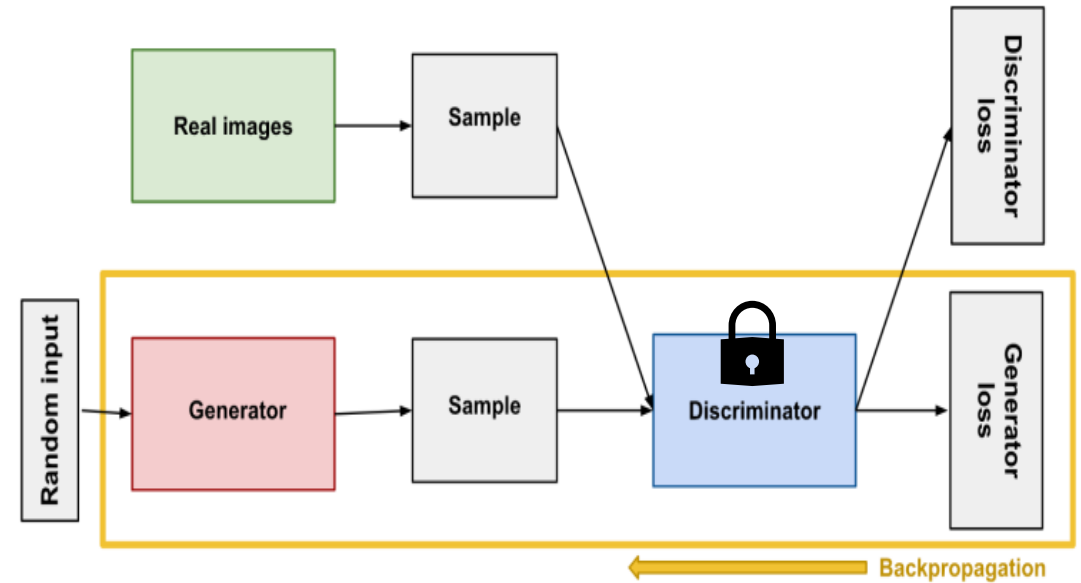
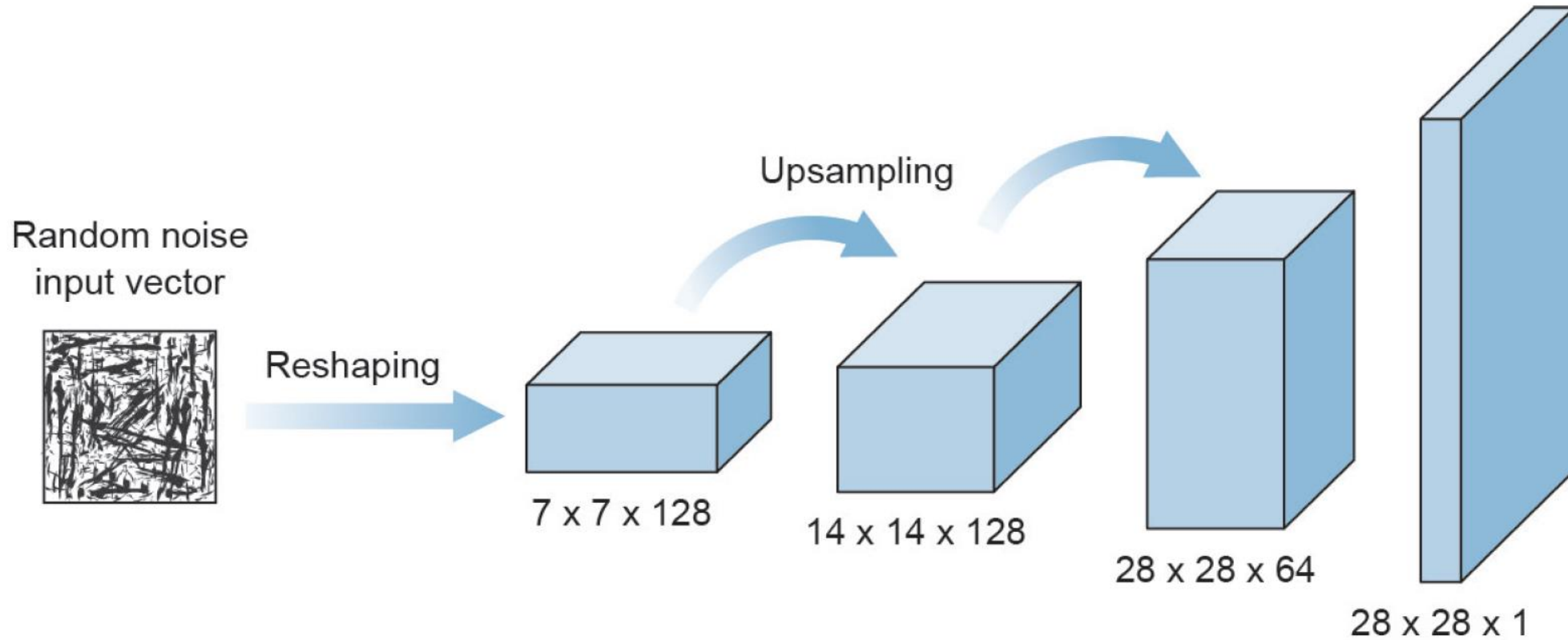


Fig: Overview of the learning process of a generator (discriminator's weights are locked)

Generator Model



Upsampling to scale features in Generator

- *Upsampling layers scale the image dimensions by repeating each row and column of its input pixels*

- *E.g.*

- *`keras.layers.UpSampling2D(size=(2, 2))`*

Input = $\begin{matrix} 1, & 2 \\ 3, & 4 \end{matrix}$

Output = $\begin{matrix} 1, & 1, & 2, & 2 \\ 1, & 1, & 2, & 2 \\ 3, & 3, & 4, & 4 \\ 3, & 3, & 4, & 4 \end{matrix}$

If the scaling factor is (3, 3)

```
[ [1. 1. 1. 2. 2. 2.]  
  [1. 1. 1. 2. 2. 2.]  
  [1. 1. 1. 2. 2. 2.]  
  [3. 3. 3. 4. 4. 4.]  
  [3. 3. 3. 4. 4. 4.]  
  [3. 3. 3. 4. 4. 4.] ]
```

Loss Functions

Minimax Loss:

In the paper that introduced GANs, the generator tries to minimize the following function while the discriminator tries to maximize it.

$$\min_G \max_D V(D, G)$$

It is formulated as a minimax game, where:

- The Discriminator is trying to maximize its reward $V(\mathbf{D}, \mathbf{G})$
- The Generator is trying to minimize Discriminator's reward (or maximize its loss)

Loss Functions

$$V(D, G) = \mathbb{E}_{x \sim p(x)} [\log D(x)] + \mathbb{E}_{z \sim q(z)} [\log(1 - D(G(z)))]$$

In this function:

- $D(x)$ is the discriminator's estimate of the probability that real data instance x is real.
- $\mathbb{E}_x$ is the expected value over all real data instances.
- $G(z)$ is the generator's output when given noise z .
- $D(G(z))$ is the discriminator's estimate of the probability that a fake instance is real.
- $\mathbb{E}_z$ is the expected value over all random inputs to the generator (in effect, the expected value over all generated fake instances $G(z)$).

Evaluating GANs

- GANs differ from traditional DL models (no explicit loss for generator)
- Generator is trained via competition with discriminator
- No direct objective metric to assess training progress
- Evaluation relies on:
 - Visual quality
 - Diversity of generated images
- **Challenges**
- No standard evaluation metric
- Difficult to:
 - Select best model during training
 - Decide stopping point
 - Compare architectures
 - Tune hyperparameters

Manual Evaluation

- Human annotators assess image quality
- Method:
 - Annotators distinguish real vs generated images
 - Used platforms like Amazon Mechanical Turk (MTurk)
- Limitations:
 - Subjective and inconsistent
 - Influenced by feedback and annotator bias

Inception Score (IS)

- Uses pretrained Inception network
- Evaluates:
 - **Image Quality** → High confidence predictions
 - **Diversity** → Uniform class distribution
- Interpretation:
 - Higher IS = Better quality + diversity
- Limitation:
 - Does not compare with real images

Fréchet Inception Distance (FID)

- Proposed by Martin Heusel et al. (2017)
- Measures distance between:
 - Real image distribution
 - Generated image distribution
- Based on:
 - Feature embeddings from Inception model
 - Fréchet (Wasserstein-2) distance
- Interpretation:
 - Lower FID = More realistic images
- Key Considerations
 - Requires large sample size (~50,000 images)
 - Small samples → unreliable (high variance, overestimation)
 - Captures both:
 - Quality
 - Statistical similarity to real data

Evaluation Approaches

Metric	Measures	Strength	Limitation
IS	Quality + Diversity	Simple	No real data comparison
FID	Distribution similarity	More robust	Needs large samples

- Domain-specific metrics
- GAN-train & GAN-test: (Proposed by Konstantin Shmelkov et al.)
 - GAN-train → Measures **recall (diversity)**
 - GAN-test → Measures **precision (quality)**
- Practical Recommendations:
 - Combine multiple evaluation methods:
 - Manual inspection
 - IS / FID scores
 - Save generated samples during training
 - Perform post-hoc model selection: Use domain-specific metrics when applicable

Evaluating GANs

- No single best GAN evaluation metric
- IS and FID are widely used but incomplete
- Manual evaluation still important
- Evaluation should consider:
 - Image quality
 - Diversity
 - Realistic metric

GAN Variations

- Progressive GANs
- Conditional GANs
- Image-to-Image Translation
- Cycle GAN
- Text-to-Image Synthesis
- Super-resolution
- Face Inpainting
- Text-to-Speech

GAN Variations – Face Impainting

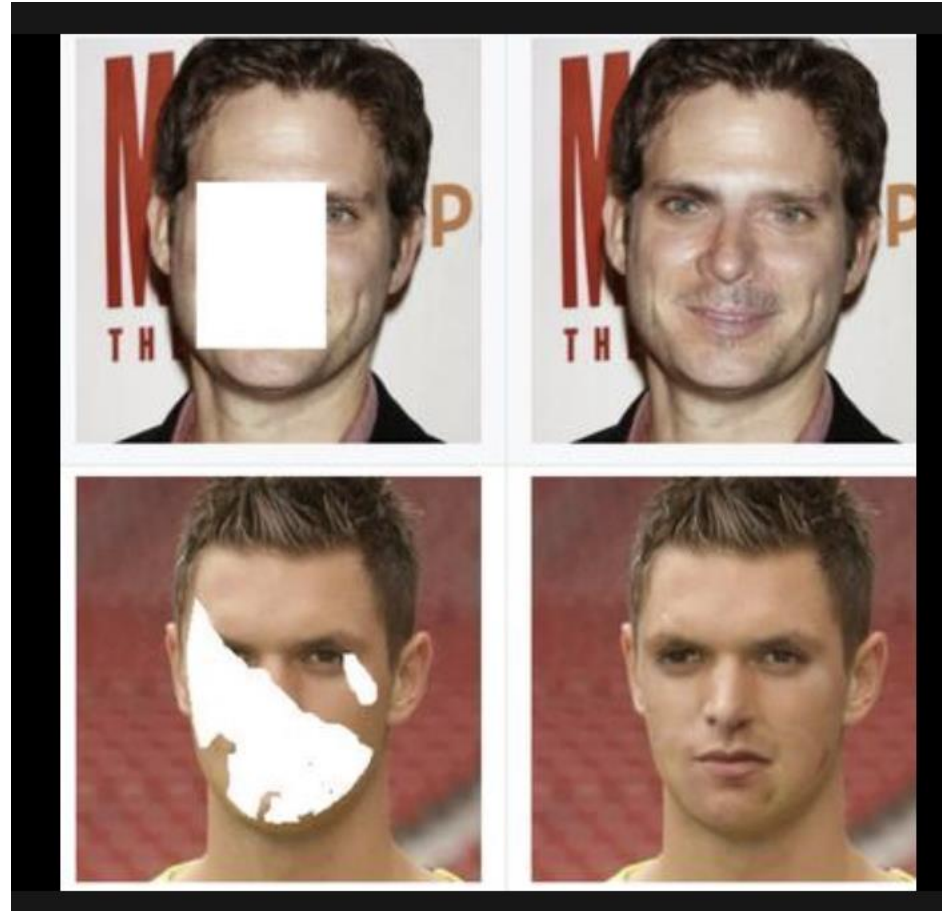


Image-to-Image Translation with Conditional Adversarial Networks



GAN Variations – Cycle GAN



Common problems with GANs

- Vanishing Gradients

If your discriminator is too good, then generator training can fail due to vanishing gradients. In effect, an optimal discriminator doesn't provide enough information for the generator to make progress.

Attempts to Remedy:

- 1) **Wasserstein loss:** The Wasserstein loss is designed to prevent vanishing gradients even when you train the discriminator to optimality.
- 2) **Modified minimax loss:** The [original GAN paper](#) proposed a [modification to minimax loss](#) to deal with vanishing gradients.

Common problems with GANs

- **Mode Collapse:** when the generator learns to produce only a limited variety of outputs, often repeating the same or very similar samples—regardless of different input noise vectors.
- **Generator exploits discriminator weaknesses:** Finds a few samples that consistently trick the discriminator.
- **Imbalance in training:** If the discriminator is too strong or too weak, training becomes unstable.
- **Non-convergence of GAN objective:** GANs don't optimize a single stable loss like traditional models.
- **Lack of diversity constraint:** No explicit penalty for producing similar outputs.

Attempts to Remedy:

- 1) **Wasserstein loss:** The [Wasserstein loss](#) alleviates mode collapse by letting you train the discriminator to optimality without worrying about vanishing gradients. If the discriminator doesn't get stuck in local minima, it learns to reject the outputs that the generator stabilizes on. So, the generator has to try something new.
- 2) **Unrolled GANs:** [Unrolled GANs](#) use a generator loss function that incorporates not only the current discriminator's classifications, but also the outputs of future discriminator versions. So, the generator can't over-optimize for a single discriminator.

Common problems with GANs

- Failure to Converge

GANs frequently fail to converge, as discussed in the module on [training](#).

Attempts to Remedy:

Various forms of regularization have been used to improve GAN convergence, including:

- 1) Adding noise to discriminator inputs**
- 2) Penalizing discriminator weights**

References

1. <https://blog.floydhub.com/gans-story-so-far/>
The evolution of GANs
2. https://developers.google.com/machine-learning/gan/gan_structure
Google
3. <https://neptune.ai/blog/generative-adversarial-networks-gan-applications>
Architecture and training of GANs
4. <https://colab.research.google.com/github/tensorflow/docs/blob/master/site/en/tutorials/generative/dcgan.ipynb#scrollTo=jQ1tEQCwxRx>
A tutorial by Tensorflow to generate fake MNIST images from random noise using Deep Convolutional Generative Adversarial Network(DCGAN)